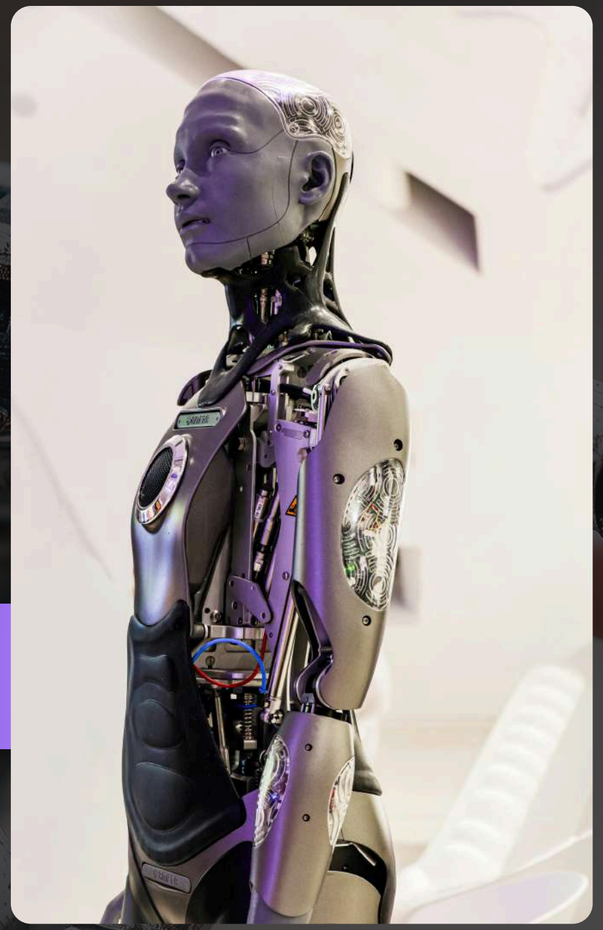


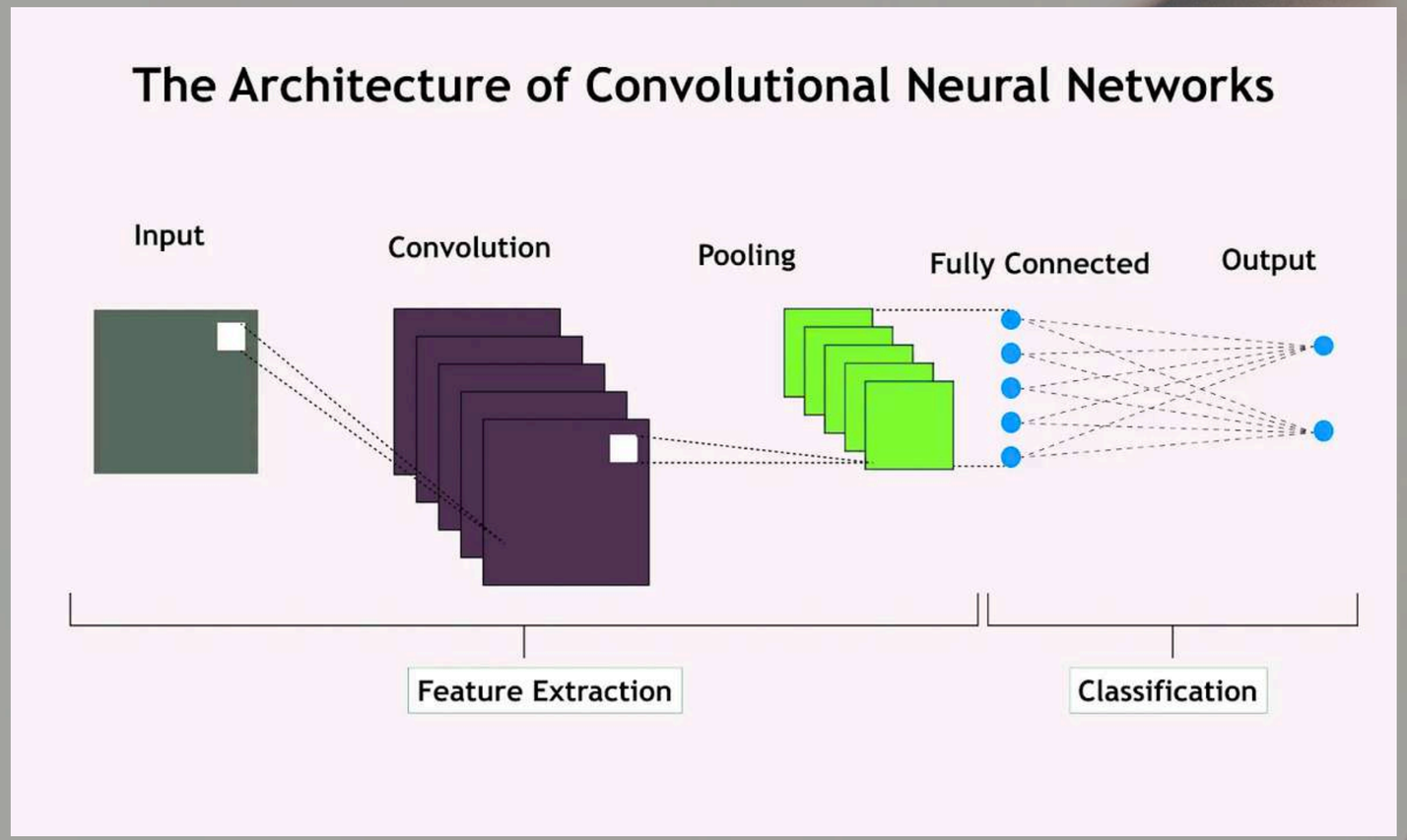
UNIT 3

Deep Learning

Convolution Neural Network

Architecture Of CNN

Diagram



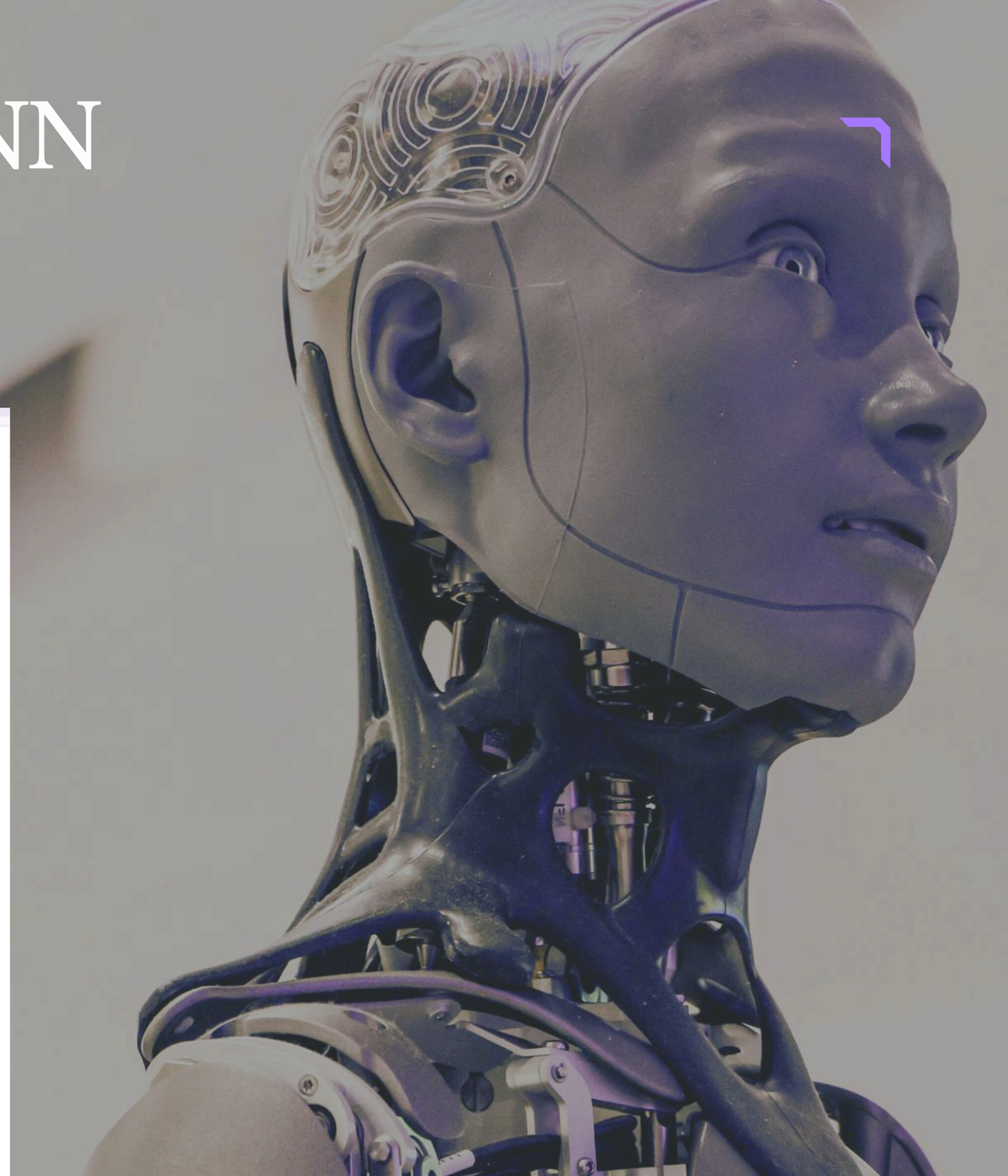
Architecture Of CNN

Diagram

$$\begin{bmatrix} 2 & 4 & 6 \\ 3 & 5 & 7 \\ 1 & 8 & 9 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

$$\begin{aligned} &(2 \times 1) + (4 \times 0) + (6 \times -1) \\ &+(3 \times 1) + (5 \times 0) + (7 \times -1) \\ &+(1 \times 1) + (8 \times 0) + (9 \times -1) \\ &= 2 + 0 - 6 + 3 + 0 - 7 + 1 + 0 - 9 \\ &= -16 \end{aligned}$$



Architecture Of CNN

Diagram

$$\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 7 & 2 & 8 & 1 \\ 0 & 9 & 3 & 5 \end{bmatrix}$$

First block:

$$\begin{bmatrix} 1 & 3 \\ 5 & 6 \end{bmatrix}$$

Maximum = 6

Second block:

$$\begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix}$$

Maximum = 4

Final pooled output:

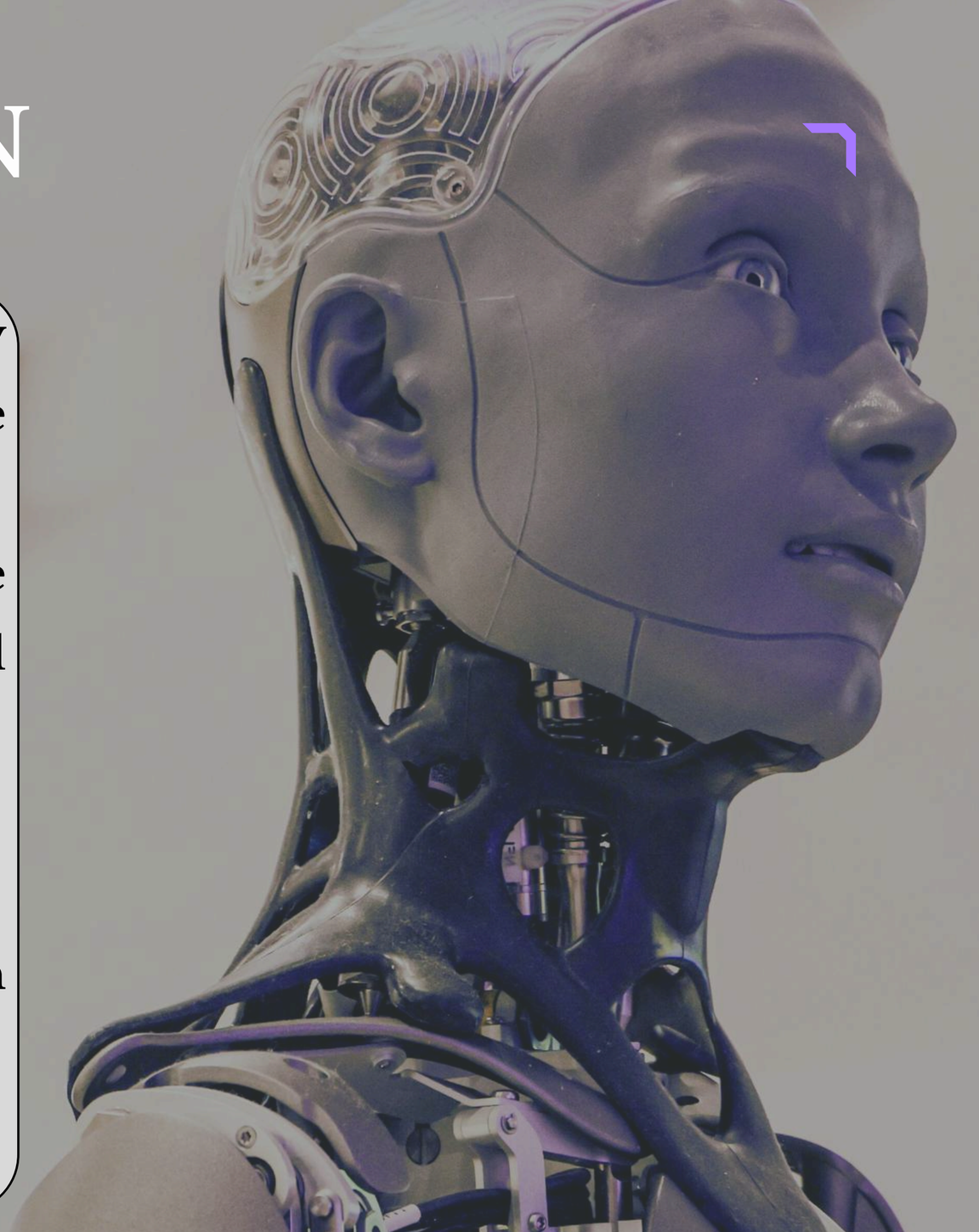
$$\begin{bmatrix} 6 & 4 \\ 9 & 8 \end{bmatrix}$$

[6,4,9,8]

Architecture Of CNN

- A CNN is a deep learning model specially designed to process data with a grid-like structure, such as images.
- It is widely used in tasks like image classification, object detection, and facial recognition.

1. Input Layer
   - This layer takes an image as input.
   - The image size is usually represented as Width × Height × Channels (RGB).
   - Example: 32 × 32 × 3.

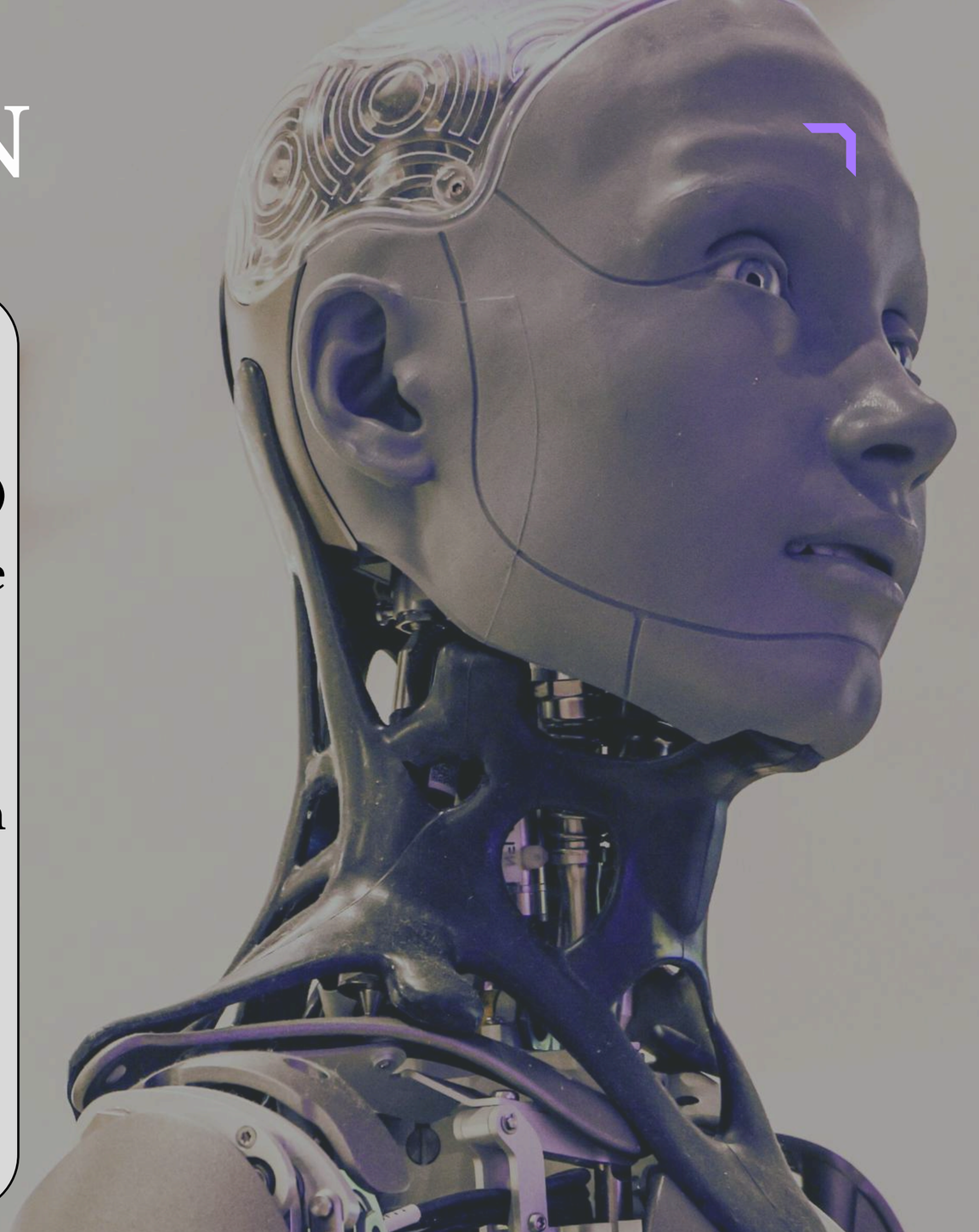


Architecture Of CNN

2. Convolution Layer

- This is the core building block of a CNN.
- It uses filters/kernels (e.g., 3×3 or 5×5 matrices) that slide over the image to detect patterns like edges or textures.
- Each filter learns to detect a specific feature.
- Multiple filters can be used in one convolution layer to extract different features.

3. ReLU (Activation) Layer

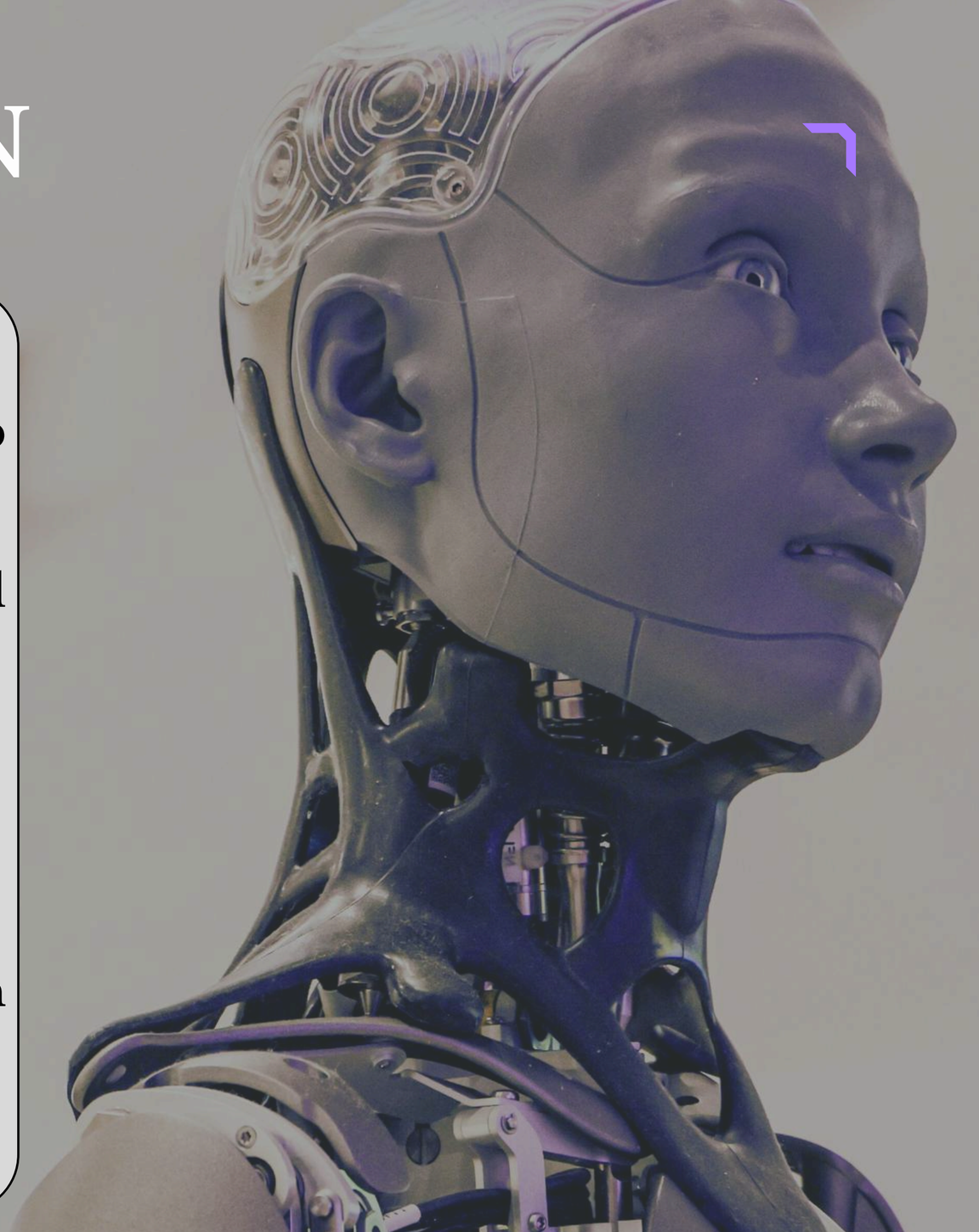


Architecture Of CNN

- ReLU stands for Rectified Linear Unit.
- It replaces negative values in the feature map with 0.
- This helps in adding non-linearity to the model and keeps only important information.

4. Pooling Layer (Subsampling / Downsampling)

- This layer reduces the size of the feature map.
- It selects the maximum value from a region (Max Pooling).



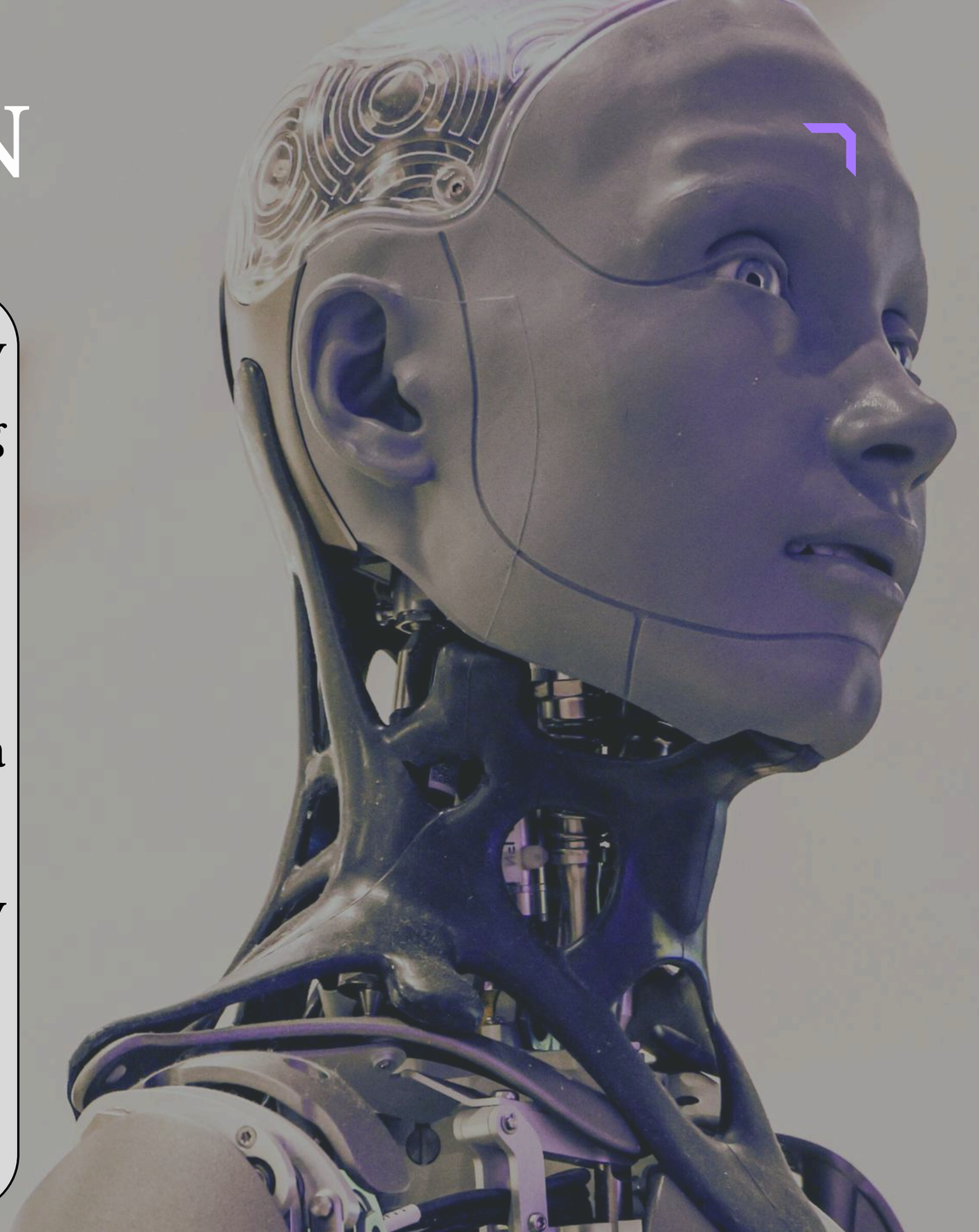
Architecture Of CNN

- Pooling makes the model computationally efficient and reduces overfitting by retaining only the most important features.

5. Flatten Layer

- This layer converts the final feature maps into a one-dimensional (1D) vector.
- The output can then be passed to the fully connected layer.

6. Fully Connected Layer

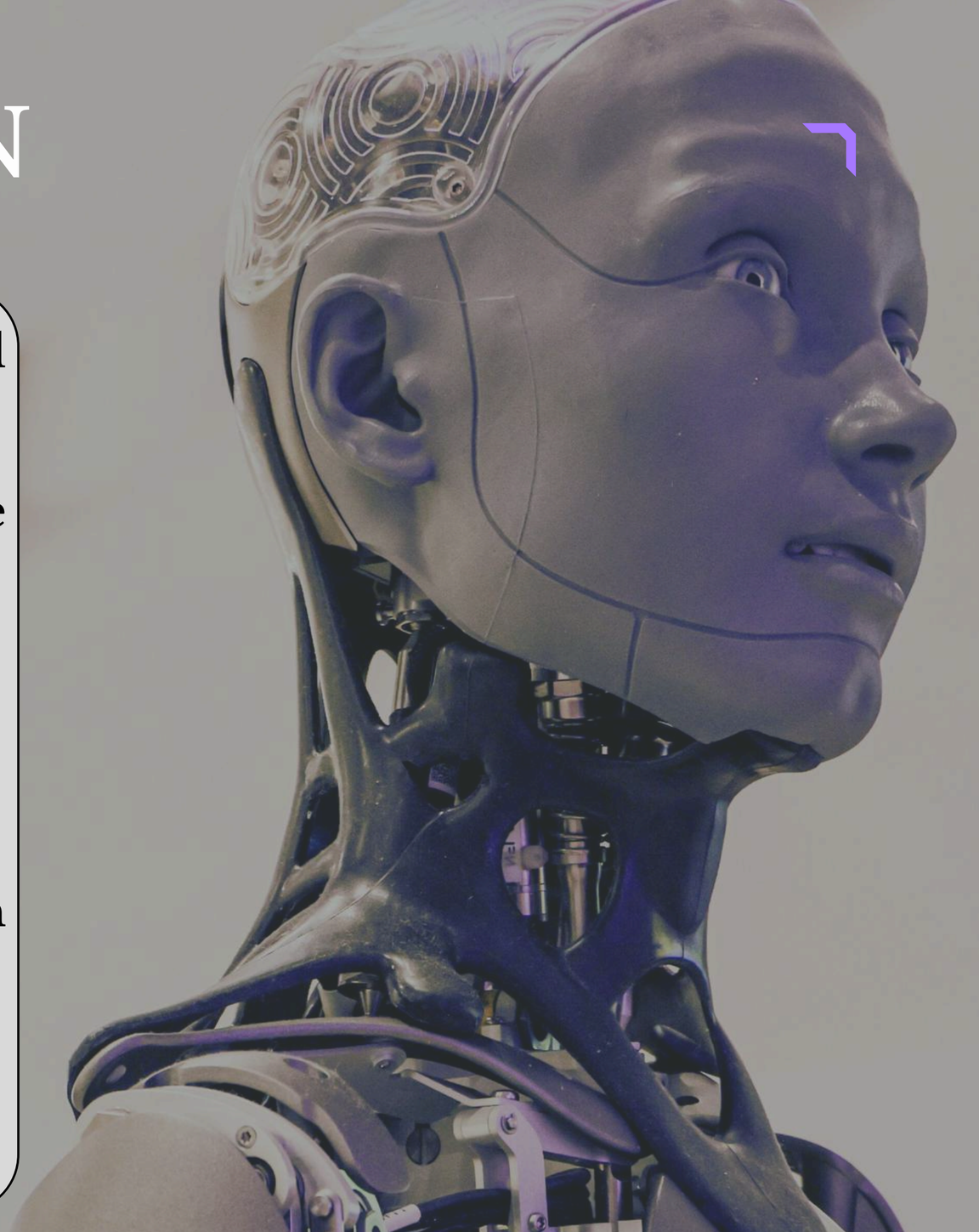


Architecture Of CNN

- In this layer, neurons are fully connected to all activations from the previous layer.
- It combines all extracted features to make predictions.

7. Output Layer

- This is the final layer of the CNN.
- It provides the prediction or classification result.



Working Of Convolution Layer

- The Convolution Layer is the heart of a Convolutional Neural Network (CNN).
- Its main job is to detect patterns/features in an image such as edges, textures, and shapes as we go deeper into the network.

1. Input Image

- Suppose we have a grayscale image of size 5×5 pixels.

Working Of Convolution Layer

2. Filter / Kernel

- A filter (also called a kernel) is a small matrix, usually of size 3 × 3 or 5 × 5.
- This filter is designed to detect specific features such as vertical edges, horizontal edges, or textures.

3. Convolution Operation

- The filter slides over the image from left to right and top to bottom.

Working Of Convolution Layer

- At each position, corresponding values of the image and filter are multiplied.
- The multiplied values are then added together.
- The result is stored in a Feature Map (also called an Activation Map).

4. Feature Map

- The feature map highlights the areas where a particular feature (such as an edge) is detected.
- The size of the feature map depends on:
  - Image size

Working Of Convolution Layer

- Filter size
- Stride (number of steps the filter moves)
- Padding

Features of Convolution Layer

i) Local Receptive Field

- Filters focus only on small regions at a time (e.g., 3 × 3).
- This helps in detecting local patterns in the image.

ii) Parameter Sharing

Working Of Convolution Layer

- The same filter is applied across the entire image.
- This reduces the number of learnable parameters and improves efficiency.

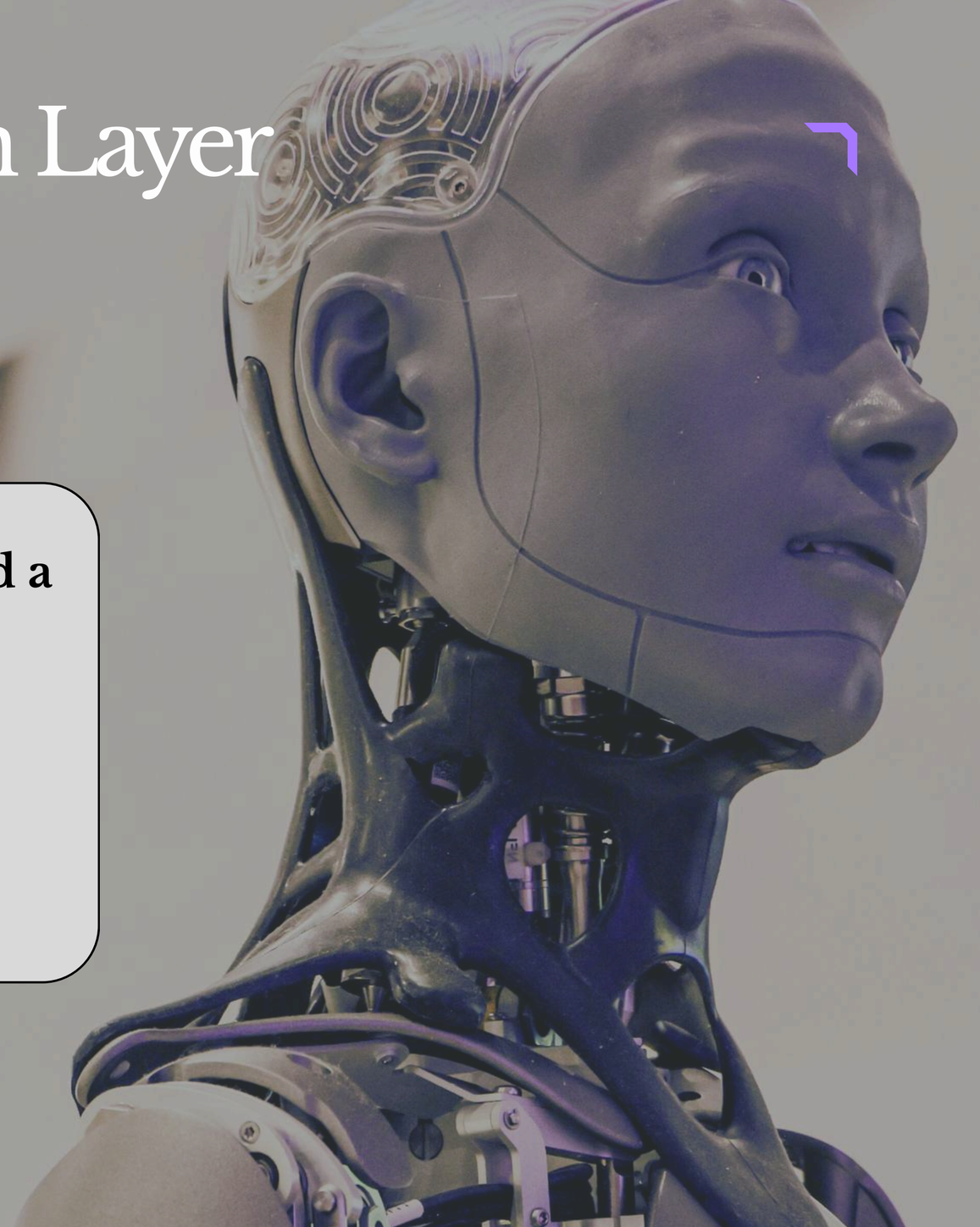
iii) Learnable Filters

- Filters are learned automatically during training.
- This helps the model adapt to the given data.

iv) Output = Feature Maps

Working Of Convolution Layer

- The output of the convolution layer is called a feature map.
- Feature maps highlight where important features are found in the image.



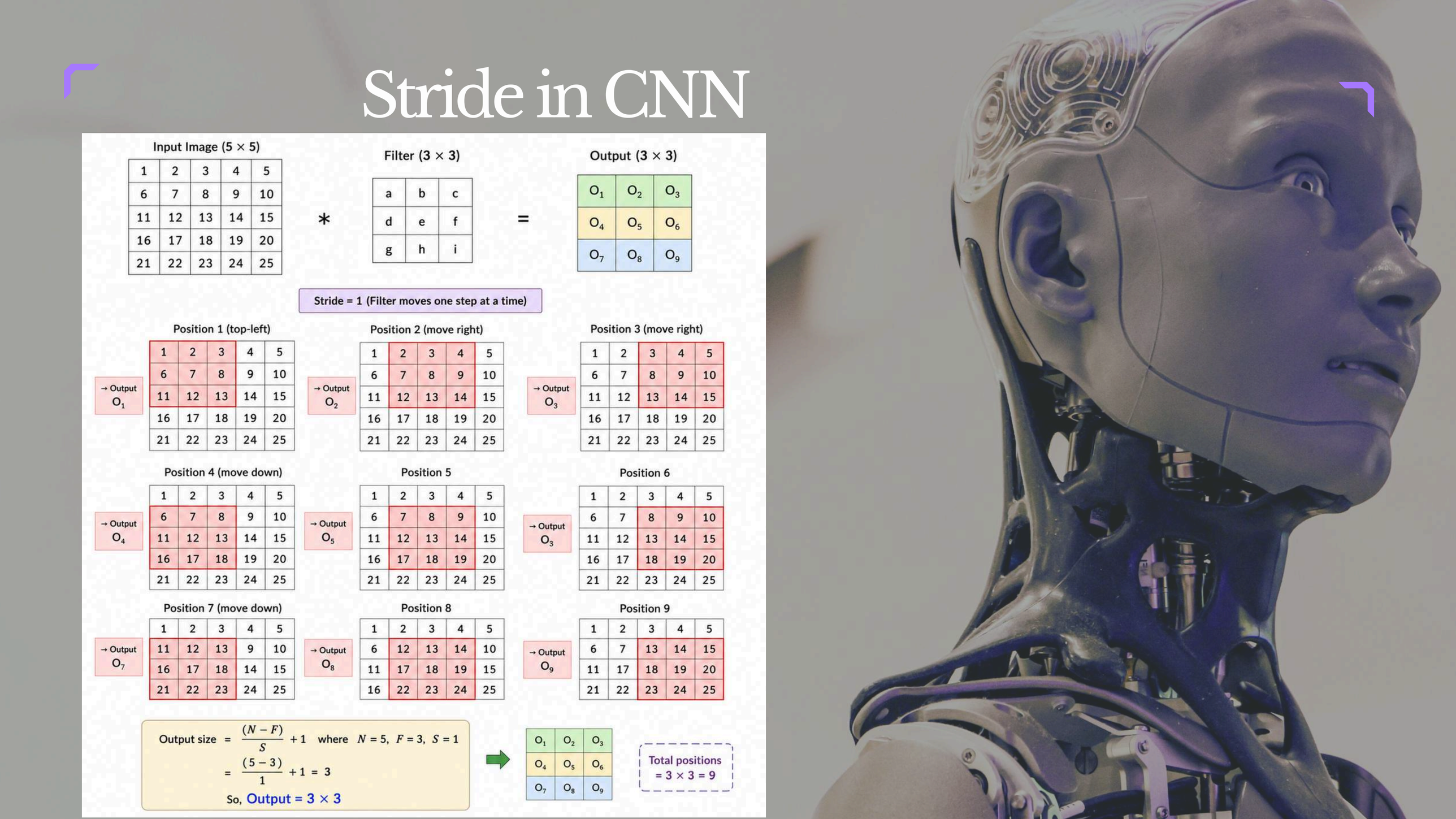
Stride in CNN

- In CNN, stride refers to how much the filter moves across the input image each time it slides.
- Normally, the filter (kernel) starts at the top-left corner of the image and moves across it from left to right and then downward.
- If the stride is 1, the filter moves one pixel at a time.
- If the stride is 2, the filter skips one pixel and moves two steps at a time.

Stride in CNN

- Using a larger stride is a simple and effective way to reduce the spatial size (width and height) of the output without using pooling.
- However, there is a trade-off:
- Larger strides can lead to loss of important details.
- This is especially important in early layers where fine-grained patterns matter the most.
- Hence, in many CNN architectures:
- Initial layers often use stride = 1

Stride in CNN



Stride in CNN

- Deeper layers may use larger strides to speed up processing
- Stride is also used in other operations like pooling, where it controls how much the pooling window moves.

Example of Stride

- Imagine a 5 × 5 input image and a 3 × 3 filter.

Stride in CNN

With Stride = 1

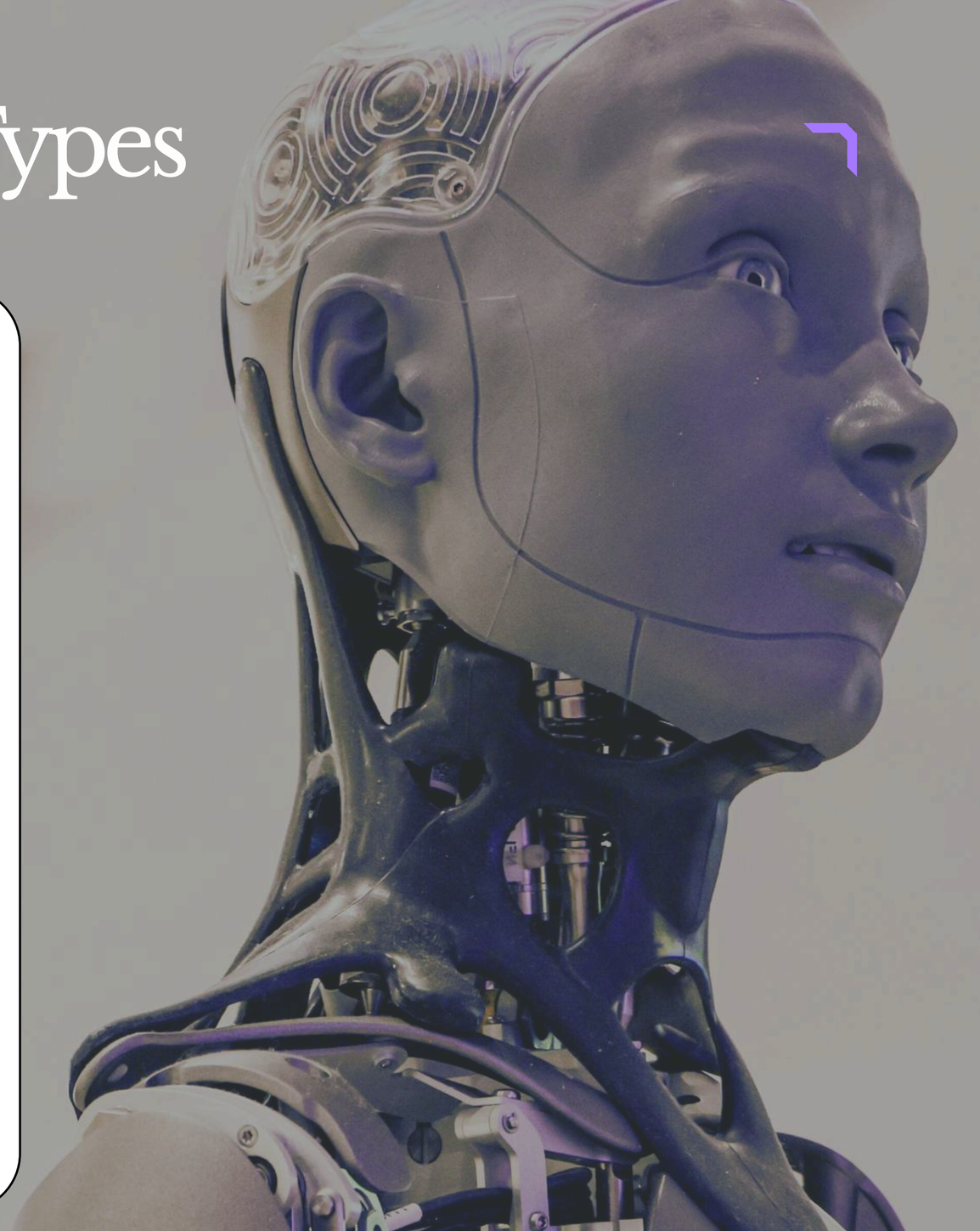
- The filter slides one step at a time.
- It moves 3 times across both width and height.
- Output size = 3 × 3.

With Stride = 2

- The filter moves two steps at a time.
- It slides faster and covers fewer positions.
- Output size = 2 × 2.

Pooling Layer and Its Types

- In CNN, the Pooling Layer is used to reduce the size (spatial dimensions) of feature maps produced by convolution layers.
- It helps in making the model faster, lighter, and less likely to overfit.
- Pooling works by summarizing or compressing information from small blocks of the feature map, usually in 2 × 2 or 3 × 3 windows, and moving that window across the feature map using stride.



Pooling Layer and Its Types

Need for Pooling Layer

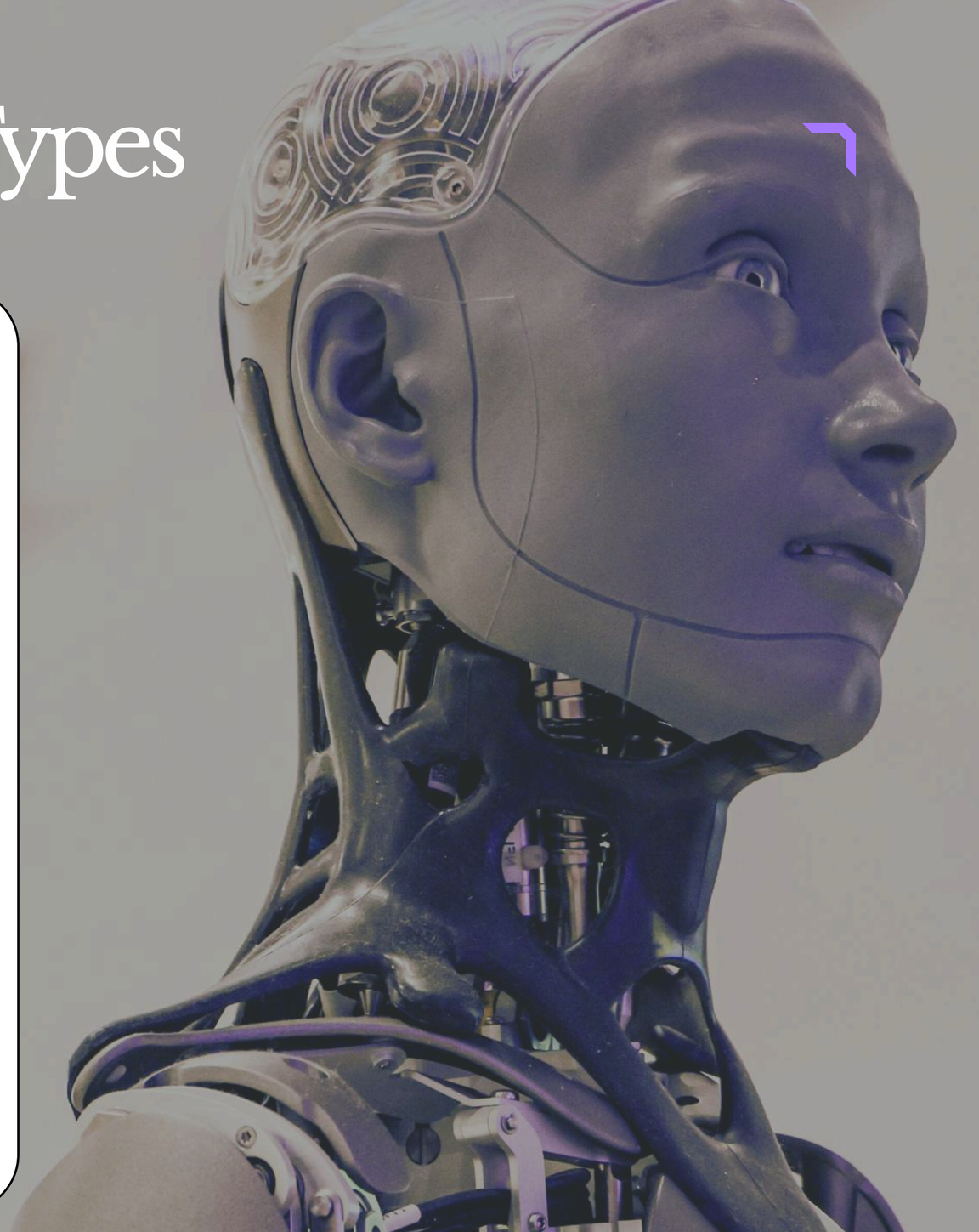
i) Reduces Dimensions

- **It decreases the size of feature maps.**
- **This makes the model faster and less memory-intensive.**

ii) Prevents Overfitting

- **Fewer parameters help avoid memorizing noise or irrelevant details.**

iii) Adds Spatial Invariance



Pooling Layer and Its Types

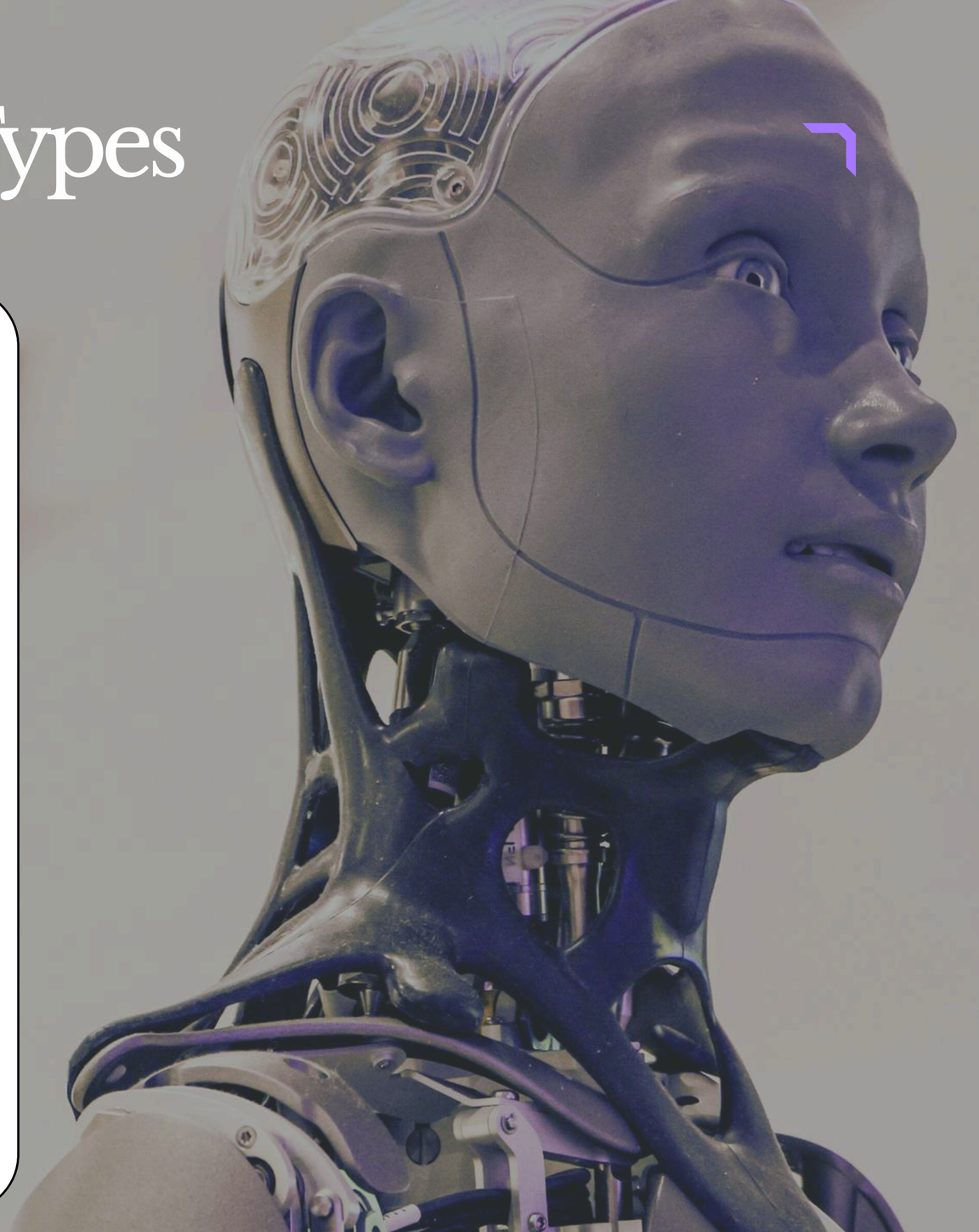
- It helps the model recognize features even when they are slightly shifted or rotated.

Types of Pooling

i) Max Pooling

- This is the most common type of pooling.
- It selects the maximum value from the pooling window, thereby keeping the most prominent feature in that region.

Example



Pooling Layer and Its Types

- From a 2 × 2 window:

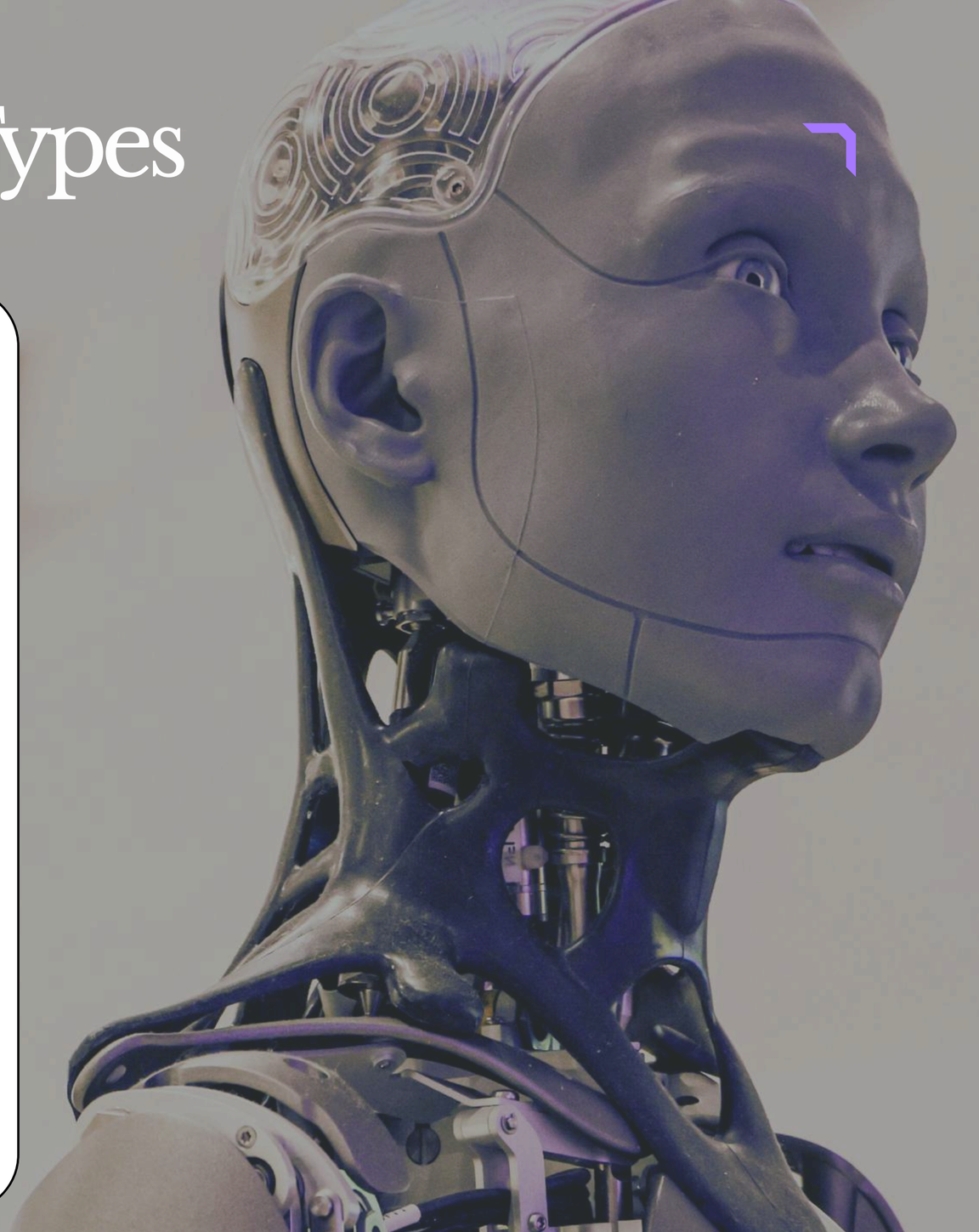
$$\begin{bmatrix} 1 & 3 \\ 2 & 0 \end{bmatrix}$$

- Max Pooling selects: 3

ii) Average Pooling

- **It calculates the average value of all elements in the pooling window.**

Example



Pooling Layer and Its Types

- From:

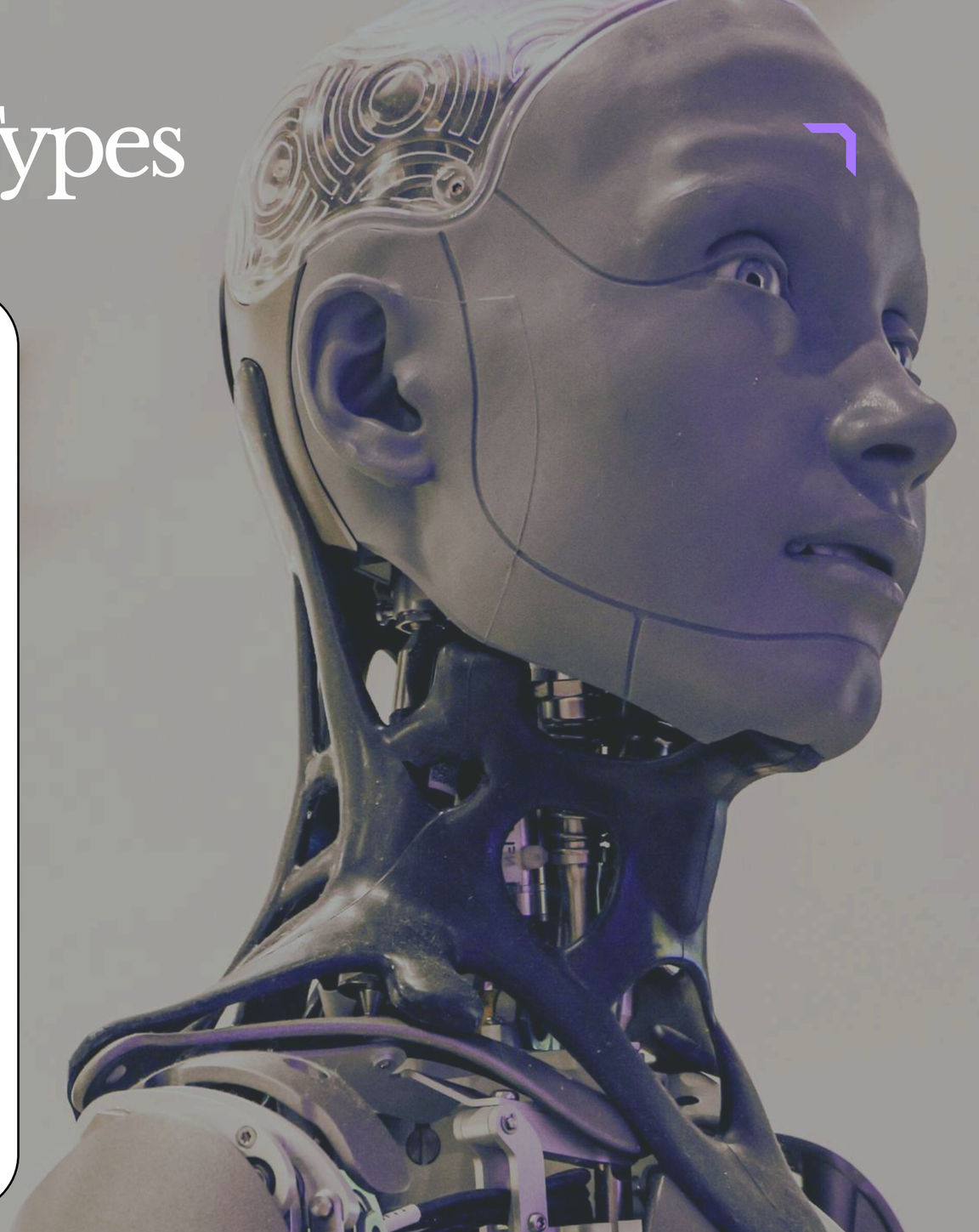
$$\begin{bmatrix} 1 & 3 \\ 2 & 0 \end{bmatrix}$$

- Average Pooling result:

$$(1+3+2+0)/4 = 1.5$$

iii) Global Pooling

- **It applies pooling over the entire feature map instead of small windows.**
- **It is often used just before the final fully connected layer to drastically reduce dimensions.**



Features of Pooling Layer

1. Dimensionality Reduction

- **Pooling reduces the width and height of feature maps.**
- **This decreases memory usage and speeds up computation.**

Example

- **A 4 × 4 feature map can become 2 × 2 after applying 2 × 2 pooling with stride 2.**

Features of Pooling Layer

2. Extracts Dominant Features

- **In Max Pooling, the most important or strongest feature in each region is selected.**
- **This helps the CNN focus on significant patterns.**

3. Translation Invariance

- **Pooling provides spatial invariance, which means the CNN becomes better at recognizing patterns (like edges) even if they are slightly**

Features of Pooling Layer

- shifted, rotated, or scaled in the image.
- This helps the model perform better on real-world images that are not perfectly aligned.

4. Reduces Overfitting

- Since pooling reduces the number of values passed to the next layers, it lowers the number of parameters in the network.
- Fewer parameters mean less chance of overfitting.

Features of Pooling Layer

5. Improves Training Speed

- **Smaller feature maps after pooling require fewer computations in deeper layers.**
- **Hence, training becomes faster and more efficient.**

6. No Learnable Parameters

- **Unlike convolution layers, pooling layers do not learn anything.**
- **There are no weights or biases.**

Padding and Its Types

- In CNN, padding means adding extra pixels (usually zeros) around the border of the input image or feature map.
- This is done before applying the convolution operation.

The purpose of padding is:

- To control the output size of the feature map
- To preserve edge information

WHAT IS PADDING?

Padding means adding extra pixels around the border of an image before convolution.

Usually these extra pixels are zeros.

WHY PADDING NEEDED?

1. **Output size control**
   Helps in controlling the size of output feature map.
2. **Edge information preserve**
   Prevents loss of important information from image borders.

PROBLEM WITHOUT PADDING

Without padding, output size reduces and edge information can be lost.

Example:

Input Image = 5 × 5
Filter = 3 × 3
Padding = 0 (No Padding)

Output = 3 × 3 (Smaller)

REAL-LIFE ANALOGY

Without padding is like cropping a photo again and again.

Original Photo

After Cropping (Loss of edges)

HOW PADDING WORKS (ZERO PADDING)

Original Image (3 × 3)

1	2	3
4	5	6
7	8	9

→

After Padding (1 pixel padding) (5 × 5)

0	0	0	0	0
0	1	2	3	0
0	4	5	6	0
0	7	8	9	0
0	0	0	0	0

OUTPUT SIZE FORMULA

$$\text{Output Size} = \frac{(N - F + 2P)}{S} + 1$$

Where,

- N = Input size
- F = Filter size
- P = Padding
- S = Stride

EXAMPLE CALCULATION

Input (N) = 5
Filter (F) = 3
Padding (P) = 1
Stride (S) = 1

$$\text{Output Size} = \frac{(5 - 3 + 2(1))}{1} + 1$$
$$= \frac{(4)}{1} + 1$$
$$= 4 + 1 = 5$$

Output Size = 5 (Same as Input)

TYPES OF PADDING

1. VALID PADDING (NO PADDING)

- No extra pixels are added.
- Filter moves only on valid positions inside the image.

Example:
Input = 5 × 5
Filter = 3 × 3
Padding = 0
Stride = 1

Input (5 × 5)

x	x	x	x	x
x	x	x	x	x
x	x	x	x	x
x	x	x	x	x
x	x	x	x	x

Output Size = 3 × 3

Output after Convolution (3 × 3)

x	x	x
x	x	x
x	x	x

Advantages
- Simple
- No unnecessary pixels added
- Computation slightly less

Disadvantages
- Edge information can be lost
- Output size reduces

Analogy: Like cropping a photo. Some part of edges gets cut.

2. SAME PADDING (ZERO PADDING)

- Zeros are added around the border.
- Padding is added so that output size = input size.

Example:
Input = 5 × 5
Filter = 3 × 3
Padding = 1
Stride = 1

Padded Input (7 × 7)

0	0	0	0	0	0	0
0	x	x	x	x	x	0
0	x	x	x	x	x	0
0	x	x	x	x	x	0
0	x	x	x	x	x	0
0	x	x	x	x	x	0
0	0	0	0	0	0	0

Output Size = 5 × 5

Output after Convolution (5 × 5)

x	x	x	x	x
x	x	x	x	x
x	x	x	x	x
x	x	x	x	x
x	x	x	x	x

Benefits
- Output size same rahta hai
  Edge information preserved
- Deep CNNs mein very useful

Analogy: Like adding a white border around photo instead of cropping.

3. FULL PADDING

- More padding is added.
- Filter is applied at every possible position including outside edges.

Example:
Input = 5 × 5
Filter = 3 × 3
Padding = 2
Stride = 1

Padded Input (9 × 9)

0	0	0	0	0	0	0	0	0
0	0	x	x	x	x	x	0	0
0	0	x	x	x	x	x	0	0
0	0	x	x	x	x	x	0	0
0	x	x	x	x	x	x	0	0
0	x	0	0	0	x	x	0	0
0	0	x	x	x	x	x	0	0
0	x	x	x	0	x	x	0	0
0	0	0	0	0	0	x	0	0

Output Size = 7 × 7

Output after Convolution (7 × 7)

x	x	x	x	x	x	x
x	x	x	x	x	x	x
x	x	x	x	x	x	x
x	x	x	x	x	x	x
x	x	x	x	x	x	x

Purpose
- To allow filter to slide over every possible position.

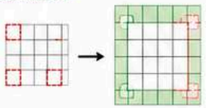
Note
- Output size becomes larger than input size.
- Less commonly used.

Analogy: Like adding extra thick border so filter can move everywhere.

EDGE INFORMATION PRESERVATION

Without padding:
- Corner/edge pixels are used less
- Edge features can be weak

With padding:
- Edge pixels get equal importance
- Important features at borders are safe



SUMMARY TABLE

Padding Type	Padding Added?	Output Size	Use Case
Valid Padding (No Padding)	No	Smaller	Simple cases, Less computation
Same Padding (Zero Padding)	Yes (Zeros)	Same	Most common in CNNs
Full Padding	More (Zeros)	Larger	Rarely used

COMPLETE UNDERSTANDING

- ✓ Without padding: output shrink hota hai, edge info lose hoti hai.
- ✓ With padding: size preserve hota hai, border features safe rehte hain, deep CNNs better kaam karte hain.

ONE-LINE DEFINITION

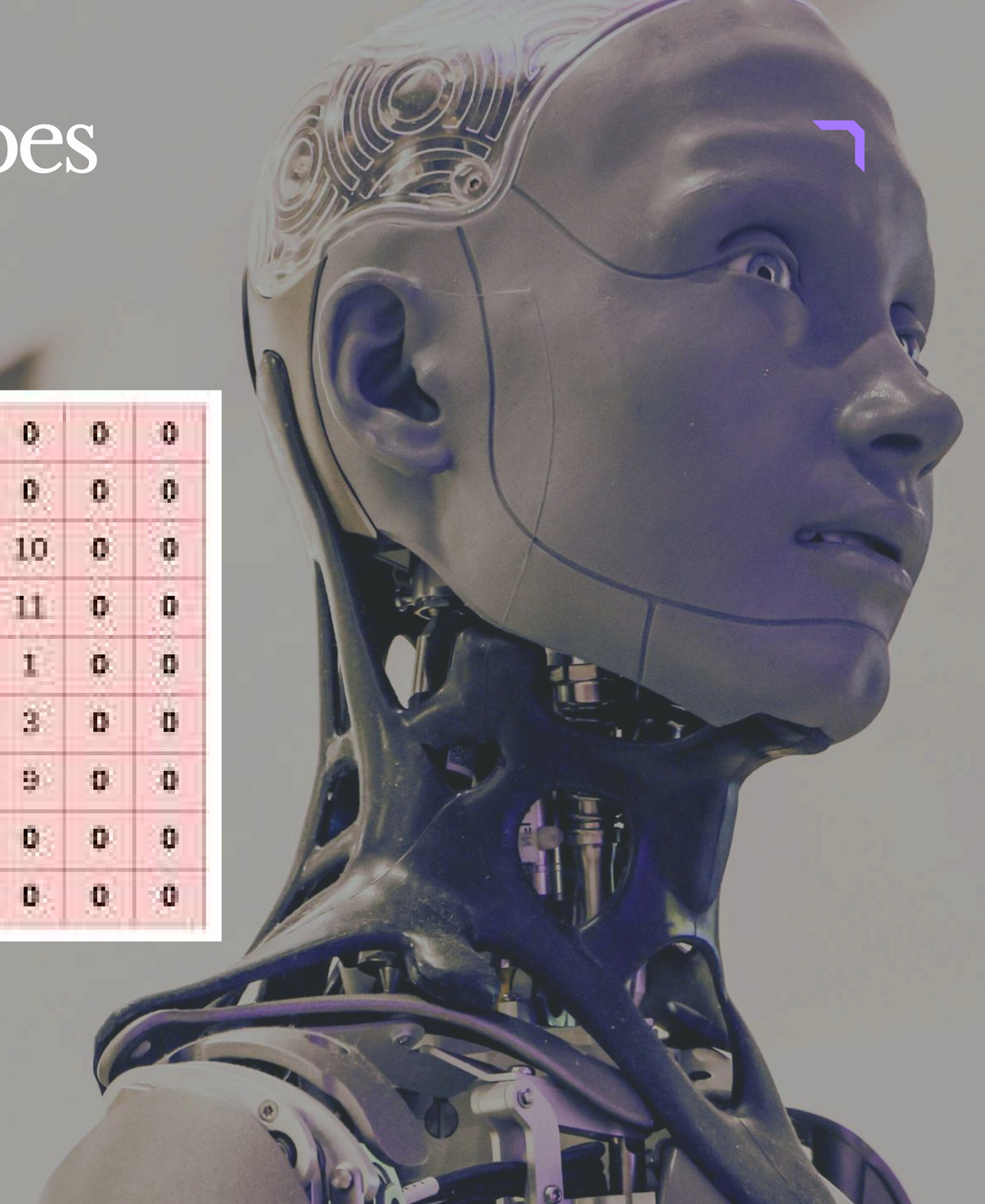
Padding image ke border ke around extra pixels add karta hai taaki output size control ho aur edge information preserve rahe.

Padding and Its Types

3	5	9	1	10
13	2	4	6	11
16	24	9	13	1
7	1	6	8	3
8	4	9	1	9

padding →

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	3	5	9	1	10	0	0
0	0	13	2	4	6	11	0	0
0	0	16	24	9	13	1	0	0
0	0	7	1	6	8	3	0	0
0	0	8	4	9	1	9	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0



Padding and Its Types

Types of Padding

i) Valid Padding (No Padding)

- In this type, no extra pixels are added.
- The convolution filter only slides inside the input image.

Example

- If we have a 5 × 5 image and use a 3 × 3 filter with no padding, the output size becomes 3 × 3.

Padding and Its Types

ii) Same Padding (Zero Padding)

- **In this type, extra pixels (usually zeros) are added around the border of the image.**
- **Padding is done in such a way that the output size remains the same as the input size.**

Example

- **A 5 × 5 image with a 3 × 3 filter and same padding gives a 5 × 5 output.**

iii) Full Padding

Padding and Its Types

ii) Same Padding (Zero Padding)

- **In this type, extra pixels (usually zeros) are added around the border of the image.**
- **Padding is done in such a way that the output size remains the same as the input size.**

Example

- **A 5×5 image with a 3×3 filter and same padding gives a 5×5 output.**

ReLU Function

- ReLU (Rectified Linear Unit) is an activation function used in CNNs and other neural networks.
- It is applied after the convolution operation or fully connected layers to introduce non-linearity into the model.
- Without an activation function like ReLU, the network would behave like a linear model no matter how many layers it has.
- The ReLU function converts all negative values into zero and keeps positive values unchanged.

ReLU Function

ReLU Formula

$$\text{ReLU}(x) = \max(0, x)$$

- If $x > 0$, it returns x.
- If $x \leq 0$, it returns 0.

Advantages of ReLU (over Sigmoid)

i) Simplicity

- ReLU is easier and faster to compute.
- It only performs a threshold operation at 0.

ii) Avoids Vanishing Gradient Problem

ReLU Function

- In functions like Sigmoid, when inputs become very high or very low, gradients become very small, which slows down learning.
- ReLU avoids this problem because it has a gradient of 1 for positive inputs.

iii) Sparse Activation

- ReLU activates only some neurons (positive ones), making the model more efficient.

ReLU Function

iv) Works Well in Practice

- Most deep learning models use ReLU because of its good performance and stability.

Disadvantages of ReLU

i) Dying ReLU Problem

- Sometimes neurons output zero for all inputs and stop learning.
- This is called the Dying ReLU Problem.
- It happens when weights are updated in such a way that they always produce

ReLU Function

- negative values, which ReLU converts into 0.

ii) Not Zero-Centered

- Unlike Sigmoid or Tanh, ReLU outputs are always positive or 0.
- This can cause uneven learning in some cases.

iii) Not Suitable for All Tasks

- In tasks like binary classification, functions such as Sigmoid or Softmax are often better for the final output layer.

Dropout Layer

- The Dropout Layer is a regularization technique used in CNNs to prevent the model from overfitting during training.
- Dropout means that during each training step, some neurons are randomly turned off (dropped).

This means they do not participate in:

- Forward pass (making predictions)
- Backward pass (updating weights)

Dropout Layer

Working of Dropout

i) During Training

- Each neuron has a probability ppp of being kept active (e.g., 0.5 or 0.7).

ii) Randomly Dropping Neurons

- The remaining neurons are randomly set to zero for that training step.

iii) Different Neurons Each Time

Dropout Layer

- In the next training step, a different set of neurons may be dropped.

iv) During Testing (Inference)

- Dropout is turned off during testing.
- Outputs are scaled appropriately to match the training process.
- The Dropout Layer is usually placed after Fully Connected Layers, but sometimes it is also used after Convolution Layers in complex CNN architectures.

Dropout Layer

Advantages of Dropout

i) Prevents Overfitting

- It forces the model not to rely too much on any single neuron.

ii) Simple and Efficient

- It is easy to implement and adds no extra learnable parameters.
- It only introduces randomness during training.

Local Response Normalization (LRN)

- Local Response Normalization (LRN) is a technique used in CNNs to normalize the output of neurons, making the learning process more stable and effective.
- It was introduced in the AlexNet architecture and is inspired by a concept from neuroscience called lateral inhibition.
- The main idea of LRN is to make strongly activated neurons stand out more clearly while suppressing weaker or less important nearby neurons.

Local Response Normalization (LRN)

Need of LRN

- During training, outputs of neurons can become very large or vary too much, especially after applying ReLU activation.
- This can make training unstable.
- LRN reduces such variations and ensures that activations remain balanced.

It helps in:

- Making learning more efficient
- Improving generalization

Local Response Normalization (LRN)

Working of LRN

- LRN takes the output of a neuron and divides it by a normalization function that depends on the activations of neighboring neurons.
- This function reduces the effect of less important neurons and boosts stronger signals.
- Mathematically, LRN uses a formula involving the squares of neighboring activations to normalize each neuron’s output.

Local Response Normalization (LRN)

Advantages of LRN

i) Enhances Important Features

- Strong activations become more prominent compared to weaker ones.

ii) Stabilizes Learning

- It prevents neuron outputs from becoming excessively large.

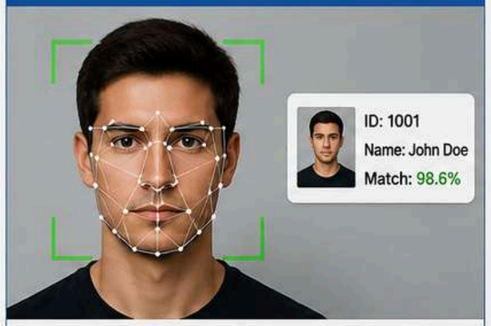
iii) Improves Generalization

- Helps the network perform better on unseen data by reducing overfitting.

Application of cnn

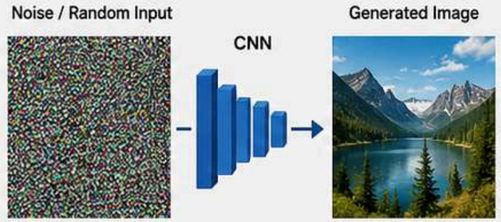
APPLICATIONS OF CNN

1. FACIAL RECOGNITION



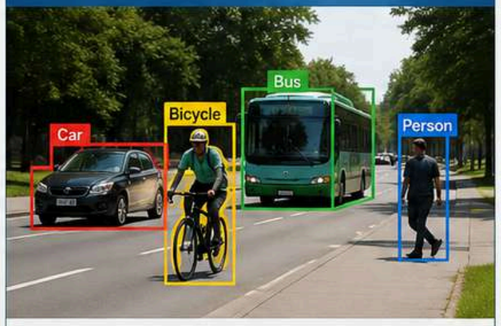
CNNs can detect and recognize human faces in images or videos.

2. IMAGE GENERATION



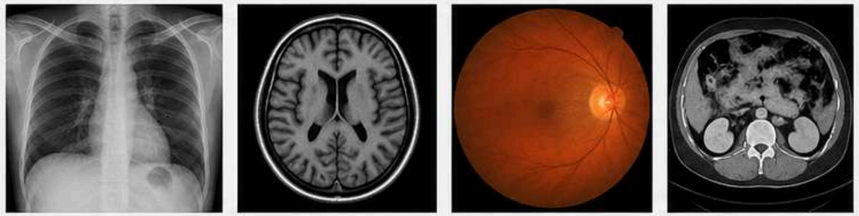
CNNs (like GANs) can generate realistic images from random noise.

3. OBJECT DETECTION



CNNs detect and locate multiple objects in an image with bounding boxes.

4. MEDICAL IMAGING

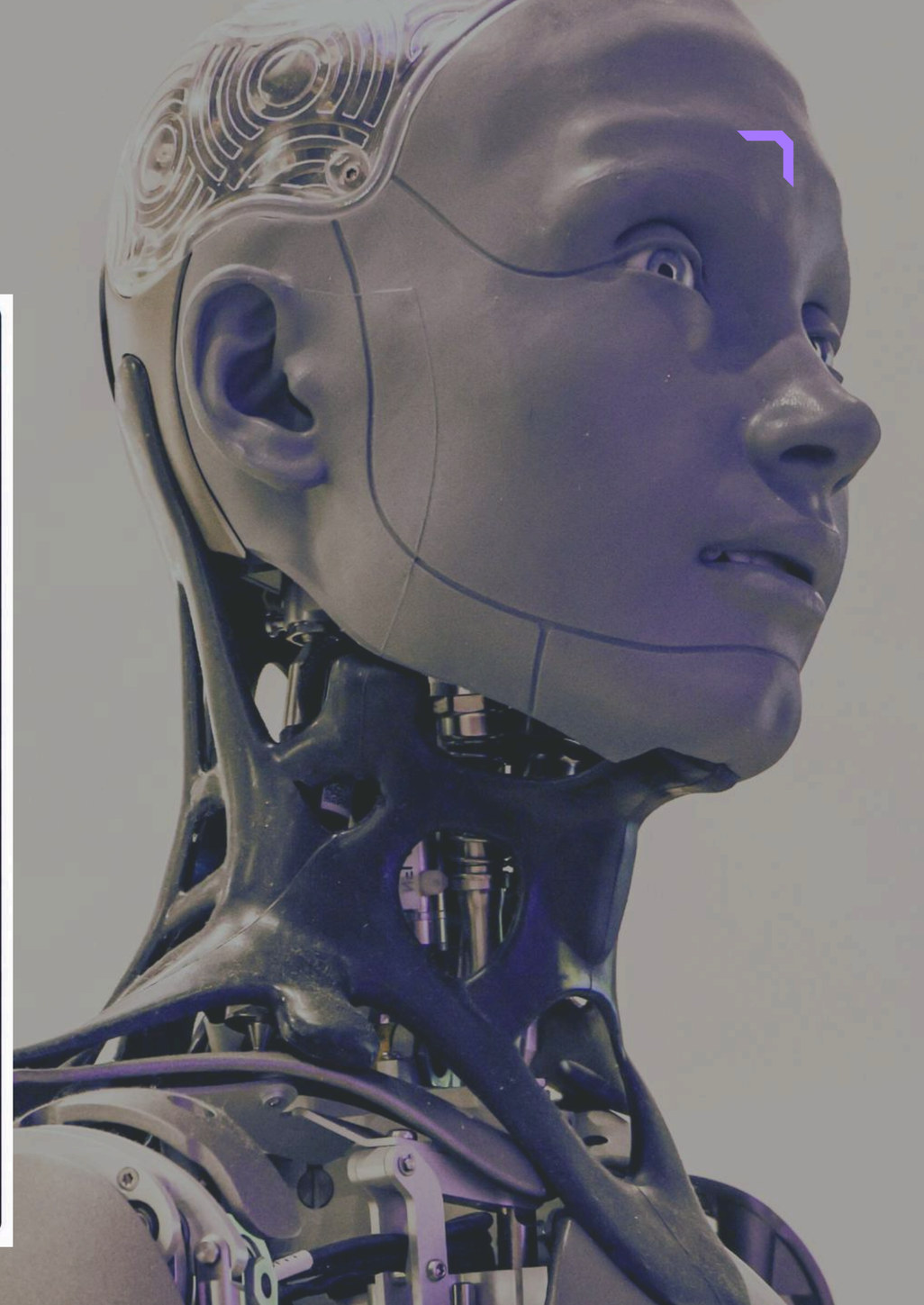


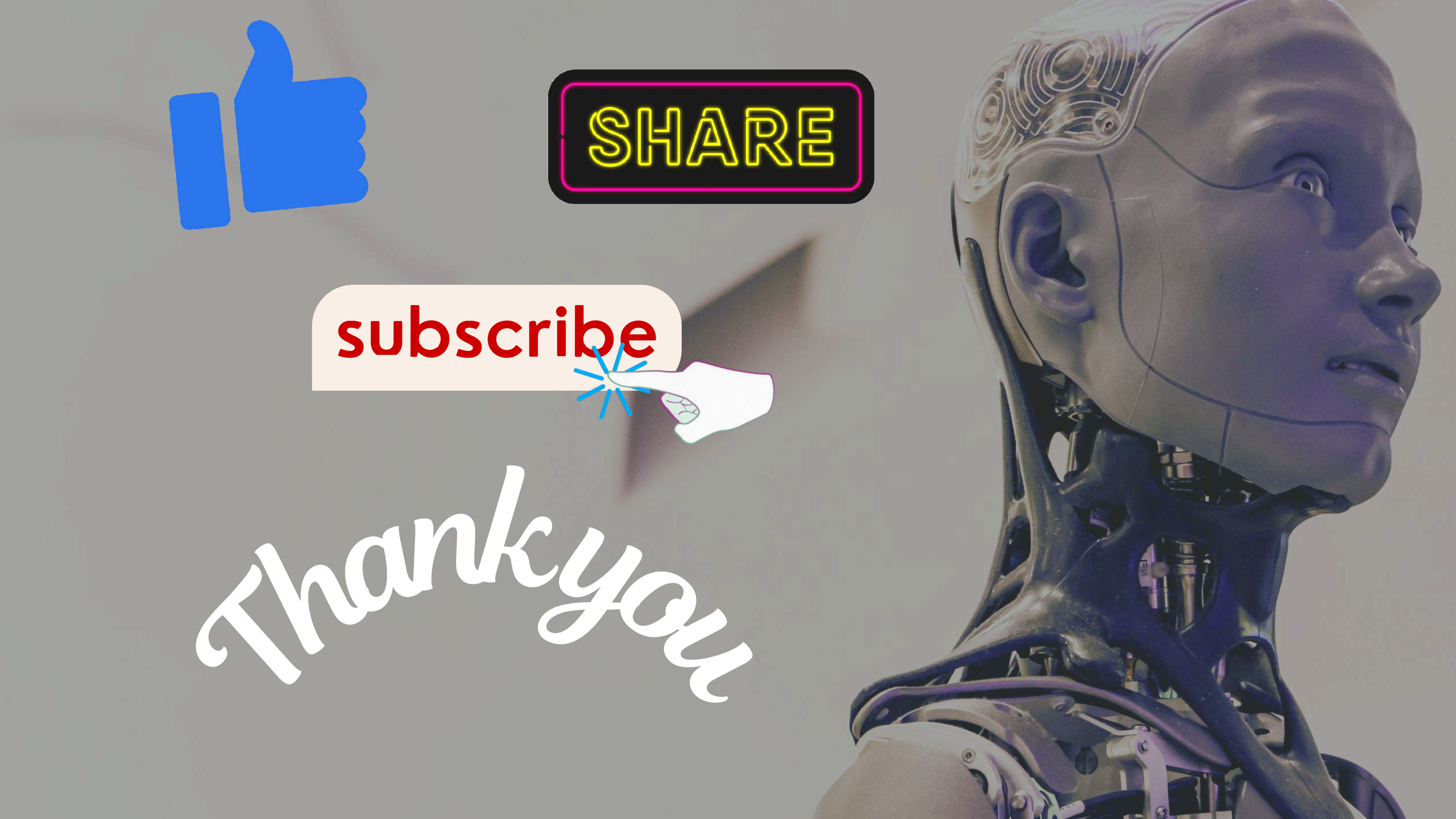
X-Ray (Lung) | MRI (Brain) | Retinal Scan (Eye) | CT Scan (Abdomen)

CNNs help in diagnosing diseases and analyzing medical scans with high accuracy.

5. OTHER APPLICATIONS

- Scene Classification
- Video Analysis
- OCR / Text Recognition
- Product Recognition
- Self-Driving Cars
- Surveillance Systems




SHARE
subscribe
Thankyou